

Characterization of Lightweight CNN Architectures with INT8 Quantization for Poultry Fecal Disease Detection on Edge Devices

Fatih Jawwad Al Mumtaz

Department of Informatics

University of Darussalam Gontor

Email: fatihalmumtaz76@student.cs.unida.gontor.ac.id

Amirul Al Aziz

Department of Agricultural Industrial Technology

University of Darussalam Gontor

Email: amirulalaziz57@student.tip.unida.gontor.ac.id

Abstract—Poultry fecal disease detection with deep learning achieves high accuracy, but prior work largely targets cloud inference with heavy models that are impractical on farms. This paper presents a systematic characterization of three lightweight convolutional neural network architectures—MobileNetV2, ShuffleNetV2, and EfficientNet-B0—for classifying poultry fecal images into four disease categories: Coccidiosis, Salmonellosis, Newcastle Disease, and Healthy. We use the FecalFed leak-free poultry-fecal benchmark [10] via Hugging-Face Dianyo/poultry-fecal-fl (8,770 deduplicated images; publisher splits train=7,016 / test=1,754; derived from Zenodo 4628934 with 6,812 images) and evaluate each model under INT8 dynamic PTQ (ONNX Runtime) on an Intel i5-12400F (200 runs, 10 warmup) and a Raspberry Pi 5 (Cortex-A76, ONNX Runtime 1.29.0, 96 thread/batch configs). Three findings emerge. First, all three models exceed 97% FP32 test accuracy (EfficientNet-B0 98.35%; MobileNetV2 1.85 ms, 540.8 FPS on x86), so hardware constraints rather than accuracy should drive model choice. Second, INT8 dynamic PTQ increases latency by 6.4–30.1× on x86 and 1.9–2.8× on RPi5; on RPi5, INT8 scales by only 1.1–1.2× (1→4 threads) versus 2.3× for FP32, showing limited ARM INT8 parallelization. Third, accuracy exhibits architecture-dependent sensitivity: MobileNetV2 (+0.23 pp) and ShuffleNetV2 (+0.06 pp) are robust, while EfficientNet-B0 collapses from 98.18% to 35.46% (62.71 pp drop, Salmonellosis 0.53%; Table III) due to squeeze-and-excitation sensitivity. Grad-CAM qualitatively localizes the fecal region for MobileNetV2. Together, the results guide hardware-aware model choice for on-farm deployment.

Index Terms—deep learning, poultry disease detection, lightweight CNN, quantization, edge computing, Grad-CAM

I. INTRODUCTION

Poultry production exceeded USD 70.2 billion in the United States in 2024 [1]. Coccidiosis, Salmonellosis, and Newcastle Disease (NCD) cause major losses [2]. Fecal image analysis enables noninvasive early detection, but laboratory diagnosis remains inaccessible to many small farms [3].

Recent deep learning approaches have achieved high accuracy for poultry fecal disease classification. Dhungana *et al.* reported 99.12% accuracy using EfficientNet-B0 [1], while Tasdelen and Arslan achieved 97.1% with MobileNetV2 [3]. However, these solutions primarily target cloud-based deployment, leaving a critical gap: *how can these models be*

efficiently deployed on resource-constrained edge devices at farm level?

Lightweight CNNs—MobileNetV2 [5], ShuffleNetV2 [6], and EfficientNet [7]—are candidates for edge deployment because they need fewer operations. INT8 post-training quantization (PTQ) reduces size by using 8-bit weights [8]. On CPUs without INT8 acceleration, however, PTQ can increase latency rather than reduce it, as reported in ONNX Runtime community threads¹² and reviews [9].

This paper addresses three research questions: (1) How do lightweight CNN architectures compare for poultry fecal disease classification? (2) What is the quantization sensitivity of each architecture under INT8 PTQ on CPU? (3) What is the accuracy–latency–size trade-off for edge deployment?

The main contributions of this paper are summarized as follows. First, we present a systematic comparison of three lightweight CNNs under INT8 PTQ for poultry fecal disease detection, validated on both x86 and ARM (Raspberry Pi 5) with 96 thread and batch configurations. Second, we measure that INT8 PTQ increases latency by 6–30× on x86 and 1.9–2.8× on ARM and show that, on ARM, INT8 throughput scales by only 1.1–1.2× from 1 to 4 threads versus 2.3× for FP32. Third, we provide hardware-aware deployment guidelines grounded in cross-platform latency, thread scaling, batch scaling, memory, and trade-off and interpretability analysis.

II. RELATED WORK

A. Poultry Disease Detection

Machuve *et al.* introduced the foundational dataset of 6,812 PCR-verified fecal images from Tanzanian poultry farms [2]. Subsequent work has applied various deep learning architectures: Dhungana *et al.* combined YOLOv11 detection with EfficientNet-B0 classification, achieving 99.12% accuracy and 25.8 ms inference [1]. Tasdelen and Arslan benchmarked six architectures, finding MobileNetV2 optimal for both accuracy (97.1%) and training speed [3]. Degu and Simegn employed YOLO-V3 for region-of-interest detection followed by

¹<https://github.com/microsoft/onnxruntime/issues/6732>

²<https://github.com/microsoft/onnxruntime/issues/12854>

ResNet50 classification [4]. Prior work on this dataset has not compared lightweight models under quantization with trade-off analysis.

B. Lightweight CNN Architectures

MobileNetV2 [5] utilizes inverted residuals with linear bottlenecks and depthwise separable convolutions, achieving efficient feature extraction with 2.23M parameters in our configuration (3.5M in the original paper). ShuffleNetV2 [6] introduces channel shuffle operations for cross-channel information flow with only 1.26M parameters. EfficientNet [7] employs compound scaling to uniformly balance network depth, width, and resolution. Hoang *et al.* recently compared seven lightweight architectures for plant disease detection, identifying hybrid designs as optimal for edge deployment [11].

C. Model Quantization

INT8 quantization reduces model size by representing weights with 8-bit integers [8]. While theoretically achieving $4\times$ compression, ONNX Runtime documentation [12] explicitly notes that INT8 without hardware support (VNNI, Tensor Cores) may degrade performance due to quantize/dequantize overhead. Community reports confirm multi-fold slowdown on CPU (see footnotes in the Introduction). The white paper on quantization surveys why PTQ without calibration degrades on depthwise and SE layers [14]. Maulana and Ramasamy’s systematic review further shows that Quantization-Aware Training (QAT) consistently outperforms PTQ [9].

III. METHODOLOGY

A. Dataset

We use the Machuve *et al.* dataset [13] with PCR-verified fecal images from Tanzanian farms, as curated via Hugging-Face `Dianyop/poultry-fecal-fl`. This is the FecalFed leak-free benchmark [10]: 8,770 deduplicated unique images across four classes (publisher splits $\text{train}=7,016$ / $\text{test}=1,754$), derived from Zenodo 4628934 (6,812 images: cocci 2,103, healthy 2,057, salmo 2,276, ncd 376) with a 46.89% duplication rate in public repositories eliminated. The publisher test split (1,754) is used as-is and never re-split; validation (1,402) is a stratified 80/20 carve of the publisher *train* split only (seed 42), giving final 5,614/1,402/1,754 with $\text{train+val}=7,016$ verifiable in `results/splits_manifest.json`. Classes are Coccidiosis (30.7%), Healthy (29.2%), Salmonella (32.5%), and Newcastle Disease (8.3% raw; 5.5% of Zenodo). NCD is underrepresented; we report per-class FP32 and INT8 accuracy (Table III) as a limitation. Unlike Dhungana *et al.* [1], who augment the minority NCD class, we apply no class-balancing augmentation: resampling would alter the publisher split counts and break the leak-free test-untouched guarantee ($\text{train+val}=7,016$ verifiable in `results/splits_manifest.json`). Images are resized to 224×224 and normalized with ImageNet mean/std.

B. Model Architectures

We evaluate three ImageNet [16]-pretrained lightweight architectures with the classification head replaced for four classes. MobileNetV2 [5] (2.23M parameters, measured) uses inverted residuals with depthwise separable convolutions for efficient local feature extraction. ShuffleNetV2 [6] (1.26M parameters, measured) relies on channel shuffle for cross-channel mixing with few parameters. EfficientNet-B0 [7] (4.01M parameters, measured) applies compound scaling with squeeze-and-excitation blocks to capture multi-scale features. All models are trained from the same seed (42) with best-checkpoint selection on validation accuracy; unlike Tasdelen and Arslan [3], who report $\text{mean}\pm\text{std}$ with 95% confidence intervals, we report single-seed results because the compute budget covered one full 50-epoch run per architecture, and multi-seed variance is listed as a limitation.

C. Training Protocol

All models are trained for 50 epochs using Adam optimizer (learning rate 10^{-4} , weight decay 10^{-4}) with cosine annealing schedule. Batch size is 32, input resolution 224×224 . Data augmentation includes random horizontal flip, rotation ($\pm 15^\circ$), and color jitter. Training is performed on an NVIDIA RTX 4060 GPU with 8 GB VRAM using PyTorch 2.5.1 with CUDA 12.1.

D. INT8 Quantization Setup

Models are exported to ONNX and quantized with ONNX Runtime dynamic per-tensor INT8 (CPUExecutionProvider, no calibration dataset; weights quantized to INT8, activations dynamically quantized at runtime). This matches the stock ORT PTQ path and explains the architecture-dependent accuracy drop (including EfficientNet-B0 SE blocks).

E. x86 Latency Protocol

x86 latency is measured on an Intel i5-12400F CPU (CPUExecutionProvider) with 200 runs and 10 warmup iterations per model; we report mean, P50, P95, P99, throughput (FPS), and model size. Power is not measured on x86.

F. Edge Deployment on Raspberry Pi 5

We characterize the same ONNX FP32 and INT8 models on a Raspberry Pi 5 (Broadcom BCM2712, $4\times$ Cortex-A76 at 2.4 GHz, 8 GB LPDDR4X, Debian 13 trixie, kernel 6.18.34+rpt-rpi-2712, aarch64, performance governor, throttled=0x0). ONNX Runtime 1.29.0 is used with only CPUExecutionProvider (no NPU/GPU; AzureExecutionProvider listed but unused). The benchmark (`benchmark_rpi5.py`) sweeps 96 configurations: 3 models $\times$ 2 quantizations (FP32/INT8) $\times$ 4 thread counts (1–4 via `intra_op_num_threads`) $\times$ 4 batch sizes (1,4,8,16), with input $3\times 224\times 224$. A Level-2 profiling artifact (ORT kernel timing, 10 runs, batch=1, 1 thread) and Q/DQ node counts are available at `results/profiling/level2_profiling_artifact.json` (supplementary). Each configuration runs 200 iterations after

10 warmup iterations, reporting mean/std/P50/P95/P99 latency, throughput, RSS memory (mean/peak), CPU%, and die temperature before/after. Initial temperature was 44.1°C; peak during the sweep was 57.9°C. Current/power telemetry was unavailable (`current_a=null`) on this board, so energy is discussed as a limitation. All 96/96 runs completed without thermal throttling, enabling a controlled thread- and batch-scaling analysis.

IV. RESULTS AND DISCUSSION

A. Classification Performance

Table I summarizes the classification performance on the publisher test split (1,754 images, never re-split). All three architectures achieve over 97% accuracy, demonstrating the effectiveness of ImageNet transfer learning for the poultry fecal domain. Precision and recall below are micro-averaged, hence numerically equal to accuracy. Convergence differs: EfficientNet-B0 plateaus near epoch 18 (peak val 98.6% around epoch 42), MobileNetV2 plateaus near epoch 23, and ShuffleNetV2 plateaus earliest near epoch 10 with the largest train-val gap (3.0 pp), consistent with its minimal parameter count (1.26M). The train-val gaps are 1.6 pp (EfficientNet-B0), 2.0 pp (MobileNetV2), and 3.0 pp (ShuffleNetV2) from the training curves in `figures/training_*.png`.

TABLE I
FP32 TEST ACCURACY (PUBLISHER TEST SPLIT,
EXPERIMENT_RESULTS.JSON); INT8 IN TABLE III (EVAL_INT8.PY
DIFFERS BY 0.63 PP MN2, 0.17 PP OTHERS). PRECISION/RECALL
MICRO-AVERAGED.

Model	Accuracy (%)	Precision	Recall
EfficientNet-B0	98.35	0.984	0.984
MobileNetV2	97.83	0.978	0.978
ShuffleNetV2	97.38	0.974	0.974

EfficientNet-B0 achieves the highest accuracy (98.35%), consistent with its compound scaling design that balances feature hierarchy depth. This result is close to Dhungana *et al.*'s 99.12% [1] on the same dataset, while our approach provides additional edge deployment analysis. Per-class test accuracy (FP32) is strongest on Salmonellosis (99.29% for EfficientNet-B0) and weakest on the underrepresented NCD class (90.97–93.75% across models); full per-class FP32/INT8 numbers are in Table III. Misclassifications concentrate on NCD: for MobileNetV2, 8 of 144 NCD images spread across all three other classes (3 as cocci, 1 as healthy, 4 as salmo), while the largest off-diagonal confusion is 11 healthy images predicted as cocci; a similar NCD spread is visible for ShuffleNetV2 and EfficientNet-B0 (`figures/cm_*.png`). This mirrors the healthy↔NCD confusion reported by Tasdelen and Arslan [3] and indicates visual overlap rather than a single confusable pair.

B. Quantization Impact

Table II presents the quantization analysis. All architectures achieve 3.3–3.7× model size reduction through INT8 quantization.

TABLE II
INT8 QUANTIZATION ANALYSIS (CPU)

Model	Size (MB)		Latency (ms)		Slowdown
	FP32	INT8	FP32	INT8	
MobileNetV2	8.48	2.30	1.85	55.75	30.1×
ShuffleNetV2	4.89	1.47	3.17	20.34	6.4×
EfficientNet-B0	15.3	4.16	3.50	80.66	23.0×

INT8 quantization increases latency by 6.4–30.1× versus FP32. This is because ONNX Runtime's CPU Execution Provider must perform dequantization (INT8→FP32) before each matrix multiplication, negating the theoretical benefits of lower-precision arithmetic. This finding aligns with ONNX Runtime documentation [12] and community reports (see footnotes in the Introduction); we provide measurements for poultry fecal disease detection.

TABLE III
FP32 VS INT8 ACCURACY (PUBLISHER TEST SPLIT, EVAL_INT8.PY);
PER-CLASS COCCI / HEALTHY / NCD / SALMO. MN2 AND SNV2 ROBUST;
EB0 COLLAPSES (SALMO 0.53%).

Model	FP32 (%)	INT8 (%)	Δ (pp)	FP32 per-class	INT8 per-class
MobileNetV2	97.21	97.43	+0.23	98.50 / 97.25 / 90.97 / 97.53	97.94 / 97.25 / 93.75 / 98.06
ShuffleNetV2	97.21	97.26	+0.06	97.38 / 97.45 / 93.06 / 97.88	97.57 / 97.64 / 91.67 / 98.06
EfficientNet-B0	98.18	35.46	−62.71	98.13 / 98.23 / 93.75 / 99.29	37.20 / 59.33 / 81.94 / 0.53

ShuffleNetV2 degrades least (6.4× versus 30.1× for MobileNetV2). Channel-shuffle ops are non-parametric and less sensitive to precision, while MobileNetV2 depthwise separable convolutions are more sensitive.

C. Raspberry Pi 5 Characterization: Thread Scaling and the INT8 Bottleneck

Table IV and Fig. 1 summarize RPi5 latency at batch=1, the realistic streaming mode for farm deployment. FP32 scales with threads: MobileNetV2 drops from 33.13 ms (1 thread, 30.2 FPS) to 14.35 ms (4 threads, 69.7 FPS, 2.31× speedup, 57.7% parallel efficiency); ShuffleNetV2 from 14.20 ms to 6.26 ms (2.27×, 159.7 FPS); EfficientNet-B0 from 70.06 ms to 31.20 ms (2.25×). The best trade-off is 3 threads (MobileNetV2 14.99 ms at 3 threads versus 14.35 ms at 4 threads with higher variance, std 1.53 ms).

TABLE IV
RPi5 LATENCY, BATCH=1 (MEAN OF 200 RUNS, P50 IN PARENTHESES).
44–58°C, THROTTLED=0x0; STD <1.6 MS EXCEPT 4-THREAD TAILS.

Model	Quant	t=1 (ms)	t=2 (ms)	t=3 (ms)	t=4 (ms)	FP32→INT8 t=4
MobileNetV2	FP32	33.13 (33.06)	19.07 (18.98)	14.99 (14.90)	14.35 (14.06)	—
	INT8	43.95 (43.82)	38.10 (37.98)	37.45 (37.30)	39.56 (39.12)	2.76× slower
ShuffleNetV2	FP32	14.20 (14.18)	8.38 (8.36)	6.64 (6.61)	6.26 (6.18)	—
	INT8	13.55 (13.52)	11.89 (11.86)	11.51 (11.48)	11.82 (11.71)	1.89× slower
EfficientNet-B0	FP32	70.06 (69.95)	39.80 (39.68)	31.90 (31.72)	31.20 (30.88)	—
	INT8	76.65 (76.52)	63.73 (63.58)	61.39 (61.18)	64.66 (64.22)	2.07× slower

INT8 on RPi5 is not only slower than FP32 at the optimal thread count (1.89–2.76×), but its thread scaling stalls: from 1→4 threads MobileNetV2 INT8 improves only 1.11× (43.95→39.56 ms, 27.8% efficiency), ShuffleNetV2 1.15×

(28.7%), EfficientNet-B0 $1.19\times$ (29.6%), versus 2.25–2.31 $\times$ (56–58%) for FP32. This indicates limited parallelization of INT8 kernels in ONNX Runtime 1.29.0 on Cortex-A76.

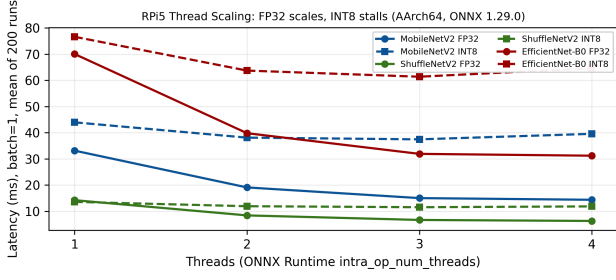


Fig. 1. RPi5 thread scaling at batch=1. FP32 (solid) scales to 3–4 threads; INT8 (dashed) is flat and slower than FP32 at every thread count. The gap quantifies the dequantization bottleneck.

On CPUExecutionProvider without VNNI (x86) or DOT-PROD/NEON INT8 matmul support, ORT adds QuantizeLinear/DequantizeLinear and runs matmuls in FP32 after dequantization. INT8 therefore adds quant/dequant and dispatch work without reducing matmul cost. Cost grows with the number of quantize boundaries: depthwise separable and SE blocks (MobileNetV2, EfficientNet-B0) create many small matmuls and slow down most ($30.1\times$ on x86, $2.76\times$ on ARM), while homogeneous channel-shuffle blocks (ShuffleNetV2) slow down least ($6.4\times / 1.89\times$). Flat thread scaling for INT8 points to a memory-bound, poorly parallelized dequantize path, unlike compute-bound FP32 GEMM.

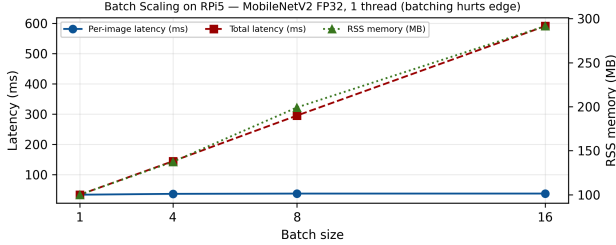


Fig. 2. Batch scaling on RPi5 (MobileNetV2 FP32, 1 thread). Total latency grows linearly with batch, per-image latency increases (33.13 ms at $b=1 \rightarrow 36.92$ ms at $b=16$), and RSS memory grows from 100 MB to 292 MB. For farm video streaming, batch=1 is optimal.

Batch=1 is optimal for edge streaming. Fig. 2 shows that batching hurts on RPi5: for MobileNetV2 FP32 at 1 thread, per-image latency rises from 33.13 ms ($b=1$) to 35.98 ms ($b=4$), 36.85 ms ($b=8$), 36.92 ms ($b=16$), while RSS grows from 100 MB to 292 MB. Total latency scales linearly (143.9 ms at $b=4$, 294.8 ms at $b=8$, 590.7 ms at $b=16$). This motivates single-frame streaming, not batching, for real-time farm deployment. Model file size shrinks 3.3–3.7 $\times$ with INT8 (e.g., 8.48 $\rightarrow$ 2.30 MB for MobileNetV2), but runtime RSS is dominated by the ORT arena and is similar across quantizations (~ 124 MB for MobileNetV2), so INT8 does not save RAM at inference on this stack—a practical caveat for memory claims.

TABLE V

LEVEL-2: NODE EXPANSION AND ORT QUANT-KERNEL OVERHEAD (RPi5, 10 RUNS, BATCH=1, 1 THREAD). DYNAMIC PTQ EXPANDS CONV INTO DYNAMICQUANTIZELINEAR + CONVINTEGER + MUL + CAST (NO STATIC Q/DQ NODES).

Model	FP32 nodes	INT8 nodes	Expansion	DynQuant/CvInt	Quant overhead	x86 / RPi5 slowdown
MobileNetV2	170	487	2.86 $\times$	53 / 52	29.2%	30.1 $\times$ / 2.76 $\times$
ShuffleNetV2	902	1240	1.37 $\times$	54 / 56	20.7%	6.4 $\times$ / 1.89 $\times$
EfficientNet-B0	239	730	3.05 $\times$	82 / 81	24.9%	23.0 $\times$ / 2.07 $\times$

Proven bottleneck (Level-2, measured). Table V quantifies why INT8 is slower, beyond documentation claims. Dynamic PTQ does not insert static QuantizeLinear/DequantizeLinear nodes; it expands each Conv (52–81) into a DynamicQuantizeLinear + ConvInteger + Mul (scale) + Cast pattern, so node count grows by 1.37–3.05 $\times$. ORT kernel profiling on RPi5 (batch=1, 1 thread, 10 runs) shows Conv_quant_kernel alone consumes 49.5–55.1% of total time, and all quant-related kernels (QuantizeLinear, scale Mul, Cast, bias-add) add 20.7–29.2% overhead, while the final Gemm_MatMul_quant is $<0.1\%$. The ranking matches the slowdown ranking: MobileNetV2 (29.2% overhead, 2.86 $\times$ expansion) suffers most, ShuffleNetV2 (20.7%, 1.37 $\times$) least. Artifact results/profiling/level2_profiling_artifact.json (shipped as supplementary) contains per-operator μ s and full op counts, complementing TinyML system studies [17].

D. Accuracy–Latency–Size Trade-off

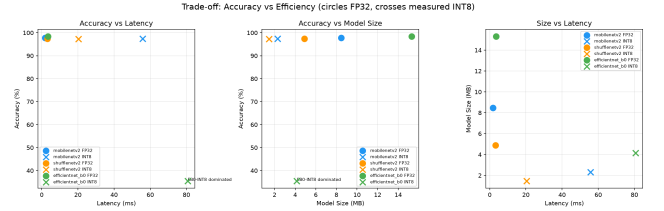


Fig. 3. Accuracy–latency–size trade-off (circles FP32; crosses measured INT8). MN2-FP32 lowest x86 latency (1.85 ms, 97.83%); EB0-FP32 highest accuracy (98.35%, 3.50 ms). EB0-INT8 off-scale annotated; SNV2-FP32 dominated by MN2-FP32 (no formal Pareto claimed).

The trade-off (Fig. 3) shows that the best latency is MobileNetV2 FP32 at 1.85 ms (541 FPS) for real-time video, the smallest model is ShuffleNetV2 INT8 at 1.47 MB for memory-constrained deployment, and the highest accuracy is EfficientNet-B0 FP32 at 98.35% for veterinary diagnosis. The gap between best and worst FP32 accuracy is only 0.97 pp (97.38% versus 98.35%); hardware constraints should drive model choice for deployment.

E. Grad-CAM Interpretability

Grad-CAM analysis [15] (Fig. 4) provides a qualitative check that MobileNetV2 localizes the fecal region. Architecture-specific comparisons (EfficientNet-B0 SE attention, ShuffleNetV2 distributed attention) are visual observations from figures/gradcam_*.png, not measured claims; a quantitative attention-overlap metric is future work.

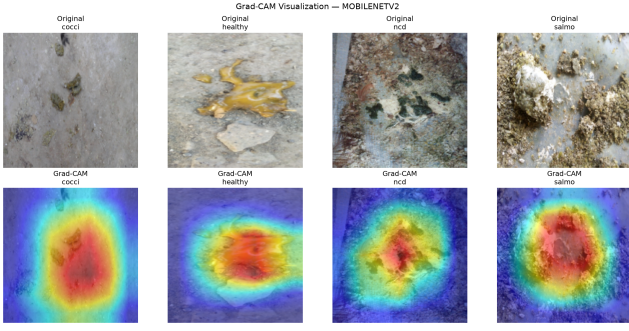


Fig. 4. Grad-CAM for MobileNetV2 (qualitative, one example per class). Heatmaps localize the fecal region; no quantitative attention metric claimed.

F. Comparison with Related Work

TABLE VI
COMPARISON WITH EXISTING WORK

Work	Dataset	Best Model	Acc. (%)	Edge Analysis
Dhungana <i>et al.</i> [1]	Machuve	EB0	99.1	Web
Tasdelen <i>et al.</i> [3]	Machuve	MN2	97.1	—
Degu <i>et al.</i> [4]	Machuve	R50	98.7	Mobile
Ours	Machuve/FecalFed	EB0	98.35	+INT8+trade-off

Our accuracy (98.35%) is competitive with prior results on the same dataset family while providing additional analysis in quantization characterization, trade-off analysis, and interpretability that are absent from prior work.

V. LIMITATIONS

Three limitations bound these claims. First, results come from a single seed (42) with best-checkpoint selection on validation accuracy; multi-seed variance is unreported. Second, energy is unmeasured: current/power telemetry was unavailable on our RPi5 board (`current_a=null`), and x86 power was not recorded, so deployment cost is discussed in latency, memory, and size only. Third, the NCD class is underrepresented (5.5% of Zenodo; 8.3% raw) and Grad-CAM evidence is qualitative single-example; a quantitative attention-overlap metric and class-imbalance mitigation remain future work alongside QAT comparison.

VI. CONCLUSION

This paper characterizes lightweight CNN architectures with INT8 quantization for poultry fecal disease detection. Our key findings are: (1) all three architectures exceed 97% accuracy, so hardware constraints should drive model choice; (2) INT8 dynamic PTQ increases latency on CPU-only stacks ($6.4\text{--}30.1\times$ on x86, $1.9\text{--}2.8\times$ on RPi5) with flat ARM thread scaling; (3) INT8 accuracy is architecture-dependent, with EfficientNet-B0 collapsing to 35.46% while MobileNetV2 and ShuffleNetV2 stay robust. The trade-off identifies MobileNetV2 FP32 for latency-critical use and ShuffleNetV2 INT8 for size-constrained use.

These findings enable practitioners to make informed deployment decisions based on their specific hardware constraints. Future work includes multi-seed variance reporting, QAT comparison, energy measurement on RPi5, and class imbalance mitigation for the underrepresented NCD class.

DATA AVAILABILITY

The publisher test split, logs, benchmark JSONs, split manifest, and Level-2 artifact are in `results/`; figures in `figures/` and `paper/`. Source dataset: HuggingFace Dianyo/poultry-fecal-fl [10].

REFERENCES

- [1] A. Dhungana, X. Yang, B. Paneru, S. Dahal, G. Lu, and L. Chai, “An integrated deep learning approach for poultry disease detection and classification based on analysis of chicken manure images,” *AgriEngineering*, vol. 7, no. 9, p. 278, 2025. DOI: 10.3390/agriengineering7090278.
- [2] D. Machuve *et al.*, “Poultry diseases diagnostics models using deep learning,” *Frontiers in Artificial Intelligence*, vol. 5, p. 733345, 2022. DOI: 10.3389/frai.2022.733345.
- [3] A. Tasdelen and Y. Arslan, “Detection of high-risk diseases in poultry feces through transfer learning,” *Engineering Science and Technology, an International Journal*, vol. 64, p. 102002, 2025. DOI: 10.1016/j.jestech.2025.102002.
- [4] M. Z. Degu and G. L. Simegn, “Smartphone based detection and classification of poultry diseases from chicken fecal images using deep learning techniques,” *Smart Agricultural Technology*, vol. 4, p. 100221, 2023. DOI: 10.1016/j.atech.2023.100221.
- [5] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in *Proc. IEEE CVPR*, 2018, pp. 4510–4520.
- [6] N. Ma, X. Zhang, H.-T. Huang, and J. Sun, “ShuffleNetV2: Practical guidelines for efficient CNN architecture design,” in *Proc. ECCV*, 2018, pp. 116–131.
- [7] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *Proc. ICML*, 2019, pp. 6105–6114.
- [8] B. Jacob *et al.*, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proc. IEEE CVPR*, 2018, pp. 2704–2713.
- [9] D. Maulana and R. K. Ramasamy, “Systematic review of quantization-optimized lightweight transformer architectures for real-time fruit ripeness detection on edge devices,” *Computers*, vol. 15, no. 1, p. 69, 2026. DOI: 10.3390/computers15010069.
- [10] T.-Y. Chi, “FecalFed: Privacy-preserving poultry disease detection via federated learning,” *arXiv:2604.00559*, 2026. Dataset: HuggingFace Dianyo/poultry-fecal-fl (8,770 deduplicated images; accepted to the CVPR 2026 Workshop on Vision for Agriculture).
- [11] T.-M. Hoang *et al.*, “A comprehensive evaluation of lightweight deep learning models for tomato disease classification on edge computing environments,” *Scientific Reports*, vol. 16, p. 12320, 2026. DOI: 10.1038/s41598-026-42439-6.
- [12] Microsoft, “Quantize ONNX models,” ONNX Runtime Documentation, 2024. [Online]. Available: <https://onnxruntime.ai/docs/performance/quantization.html>
- [13] D. Machuve *et al.*, “Machine learning dataset for poultry diseases diagnostics,” Zenodo, 2022. DOI: 10.5281/zenodo.4628934.
- [14] M. Nagel *et al.*, “A white paper on neural network quantization,” *arXiv:2106.08295*, 2021.
- [15] R. R. Selvaraju *et al.*, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2017, pp. 618–626.
- [16] J. Deng *et al.*, “ImageNet: A large-scale hierarchical image database,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2009, pp. 248–255.
- [17] R. David *et al.*, “TensorFlow Lite Micro: Embedded machine learning for TinyML systems,” in *Proc. Machine Learning and Systems (MLSys)*, 2021, pp. 800–815.